

Asignación dinámica de memoria

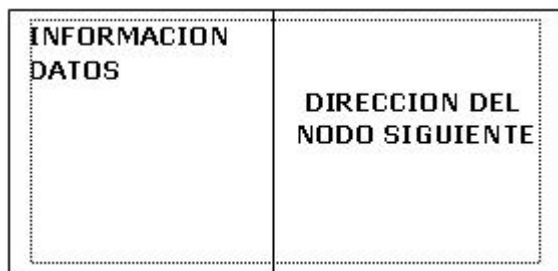
Listas enlazadas – Pilas y Colas

En pascal podemos trabajar de todo, es decir, variables, constantes, registros punteros, listas, colas, stack, así podremos manipular cada uno de los datos que se ingresan, la característica principal del pascal es tener un diseño estructurado al momento de programar y una sintaxis muy exigente ya que si un punto y coma no son colocados emitirá Error que hará que el programa no corra, además su diseño es similar a un editor de texto, y su ejecución se hace a través de pantallas del ms-dos, siendo compatible con todas las plataformas de Windows. Además tenemos distintas versiones de pascal que nos servirá para aprender. Pascal 5, 7, Turbo pascal, y Free Pascal

PUNTEROS

En este manual trataremos lo que es una cola, stack, punteros, listas enlazadas, listas doblemente enlazadas, pilas, Empezaremos a explicar que una cola, stack, lista, etc., tienen un mismo componente un NODO.

Nodo: es un elemento formado por dos partes, la parte izquierda es la información y en donde se quedan los datos y la derecha es el la dirección de la memoria del próximo elemento del nodo siguiente.



Cuando hablamos de una "Cantidad de memoria variable, utilizamos construyendo un apuntador (puntero) cuyos valores corresponden a direcciones de memoria, en las que se incluye una dirección NULA llamada NIL.

Definición.

```
TYPE Tipo_P = ^Tipo_V ;  
VAR p,q : Tipo_P
```

Son datos dinámicos que pueden crearse o eliminarse en el transcurso del programa, almacena la dirección fija en memoria, las variables dinámicas se definen y acceden a ellas a través de variables de tipo puntero

Var

```
Pi:^ integer;  
Pr:^ real;
```

a) una variable tiene asignación estática de memoria cuando su tamaño difiere en el momento de la compilación (un entero gasta 2 bytes).(vectores)

b) una variable tiene asignación dinámica de memoria cuando se define en la compilación pero no ocupa memoria (no existe realmente como variable) y crea existencia en el momento de la ejecución (punteros).

Ej.

Var

X:integer;

Pi:^ integer;

Begin

Sea x:=6

Pi^:= x;

Writeln(pi^);

Writeln(x);

End

CREACION DE UN PUNTERO

Var

p,q:^ integer;

x:integer;

begin

new(p);

new(q);

p^:=3;

q:= p;

writeln(p^,q^),

new(p);

P^:=5;

Writeln(p^, q^);

End,

Para liberar la memoria de un puntero solo debemos hacer un DISPOSE(puntero). Y quedara liberada.

LISTAS

Es una estructura de datos LINEAL con un número limitado de elementos homogéneos que se llaman nodos. Se puede acceder a estos a través de su dirección, se pueden insertar y suprimir en cualquier posición

Por convención el último nodo se marca con NIL (nulo)

La definición de la lista seria:

TYPE

PUNTERO:^NODO;

NODO = RECORD;

INFO: INTEGER;

```
SIG : PUNTERO;  
END;
```

Crear una lista

En toda creación de una lista existen dos pasos:

- a) creación del primer nodo.
- b) creación del resto de nodos.

a) creación del primer nodo

```
new(lista);  
lista^_nodo:= 1;  
lista^.siguiente=nil; .
```

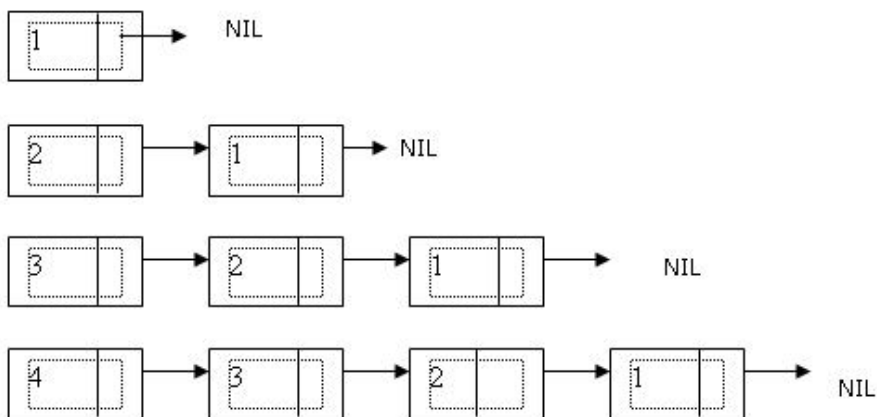
b) creación de una lista con N nodos.

```
begin  
  new(lista);  
  lista^_info:= 1;  
  aux:=lista;  
  for i:=1 to N do  
    begin  
      new(aux^.sig);  
      aux:=aux^.sig;  
      aux^.info:=i;  
    end;  
  aux^.sig:=nil;  
end;
```

Recorrido de una lista

```
Aux:= lista;  
While aux <> nil do  
  Begin  
    Write (aux^.info);  
    Aux:=aux^.siguiente;  
  end
```

Modo de Inserción por el principio de la lista.



```

For i:=n to downto 1 do
Begin
    New(aux);
    Aux^.info:=l;
    Aux^.siguiente:=lista;
    Lista:=aux;
end

```

Busqueda de un elemento dentro de una lista

```

Procedure Busqueda(comienzo:puntero, Elemento:integer);
Var
    Aux:puntero;
    Inc:boolean;
Begin
    Aux:=comienzo;
    Inc:=false;
    While (not inc) and (aux <> nil) do
        Begin
            If aux^.info: = elemento then
                Inc:=true;
            else
                Aux:=aux^.siguiente;
            End
        End
    If inc=false then
        Write(' Elemento no encontrado ')
    else
        Write(' elemento Encontrado');
    End
End

```

Borrar una lista

```

Procedure Borrar(var l:=lista, elem:=integer);
Var
    Igual, anterior:lista;
Begin
    { se debe buscar la posición del elemento a borrar }

    Actual:=l,
    Anterior:=l;
    While (actual <> nil) and (actual^.clave <> elem) do
        Begin
            Anterior:= actual;
            Actual:=actual^.siguiente;
        End;
    If (actual <> nil) and (actual^. Clave<> elem) then
        Begin
            If (anterior=actual) then
                L:=actual^. Sig { borrar el primero }

```

```

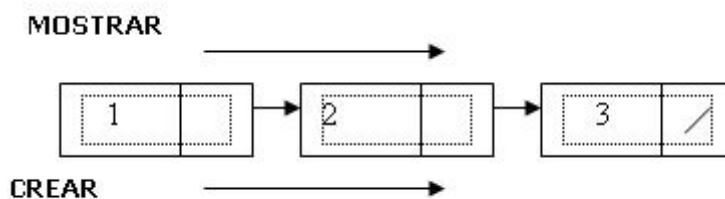
Else
  Anterior^. Sig:= actual^.sig;
  Dispose(actual);
End;
End;

```

COLAS

Una cola es una estructura de datos caracterizada por ser una secuencia de elementos en la que la operación de inserción push se realiza por un extremo y la operación de extracción pop por el otro. También se le llama estructura FIFO (del inglés First In First Out), debido a que el primer elemento en entrar será también el primero en salir. El tipo cola representa la idea que tenemos de cola en la vida real. La cola para subir al autobús está compuesta de elementos (personas), que dispone de dos extremos comienzo y fin. Por el comienzo se extraerá un elemento cuando haya comprado el billete para su viaje, y si llega una nueva persona con intención de usar el autobús, tendrá que colocarse al final y esperar que todos los elementos situados antes que él abandonen la cola.

La definición de la COLA seria:



Crear una COLA - IMPLEMENTACION

```

type
  tipocola = ^regcola;
  regcola = record
    dato:integer;
    sig:tipocola;
  end;

  var
  cola:tipocola;

```

INICIALIZAR UNA COLA

```

procedure Inicializar_cola (var c:tipocola);
begin
  c:=nil;
end;

```

VERIFICAR SI UNA COLA ESTA VACIA

```
function cola_vacia (c:tipocola):boolean;  
begin  
  if c = nil then  
    result:=true  
  else  
    result:=false;  
  end;  
end;
```

AGREGAR UN ELEMENTO A UNA COLA (ENCOLAR)

```
procedure encolar (var c:tipocola;num:integer);  
var  
  nuevo,aux:tipocola;  
begin  
  new(nuevo);  
  nuevo^.dato:= num;  
  nuevo^.sig:=nil;  
  
  if (cola_vacia(c)) then  
    c:=nuevo  
  else  
    begin  
      aux:=c;  
      while (aux^.sig <> nil) do  
        aux:=aux^.sig;  
      aux^.sig:=nuevo;  
    end;  
  end;  
end;
```

QUITAR UN ELEMENTO DE UNA COLA (DESENCOLAR)

```
function desencolar (var c:tipocola):integer;  
var  
  aux,ant:tipocola;  
  num:integer;  
begin  
  if (not cola_vacia(c)) then  
    begin  
      aux:=c;  
      ant:=aux;  
      while (aux^.sig <> nil) do  
        begin  
          ant:=aux;  
          aux:=aux^.sig;  
        end;  
      num:= aux^.dato;  
      ant^.sig:=nil;  
      dispose(aux);
```

```
    result:=num;  
end;
```

```
end;
```

MOSTRAR EL ULTIMO ELEMENTO DE UNA COLA

```
function ultimoCola (c:tipocola):integer;  
begin  
if (not cola_vacia(c)) then  
begin  
while (c^.sig <> nil) do  
c:=c^.sig;  
result:= c^.dato;  
end;
```

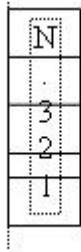
```
end;
```

CONTAR LOS ELEMENTOS QUE HAY EN UNA COLA

```
function contarCola (c:tipocola):integer;  
var  
cont:integer;  
begin  
cont:=0;  
while (c <> nil) do  
begin  
cont:= cont+1;  
c:= c^.sig;  
end;  
result:= cont;  
end;
```

STACK(PILAS)

Estructura de datos lineal de elementos homogéneos, en la cual los elementos entran y salen por el mismo extremo, llamado, TOPE, CABEZA o cima de la pila. También se denominan “listas lifo”, (last in first out) último en entrar primero en salir. Al crearse la pila su índice se inicializa en -1



INICIALIZAR UNA PILA

```
procedure Inicializar_pila (var p:tipopila);
begin
  p:=nil;
end;
```

VERIFICAR SI UNA PILA ESTA VACIA

```
function pila_vacia (p:tipopila):boolean;
begin
  if p = nil then
    result:=true
  else
    result:=false;
end;
```

AGREGAR UN ELEMENTO A UNA PILA (APILAR)

```
procedure apilar (var p:tipopila;num:integer);
var
  nuevo,aux:tipopila;
begin
  new(nuevo);
  nuevo^.dato:= num;
  nuevo^.sig:=nil;

  if (pila_vacia(p)) then
    p:=nuevo
  else
    begin
      aux:=p;
      while (aux^.sig <> nil) do
        aux:=aux^.sig;
```

```
    aux^.sig:=nuevo;
end;
end;
```

QUITAR UN ELEMENTO DE UNA PILA (DESAPILAR)

```
function desapilar (var p:tipopila):integer;
var
    aux:tipopila;
    num:integer;
begin
    if (not pila_vacia(p)) then
        begin
            num:= p^.dato;
            aux:=p;
            p:=p^.sig;
            dispose(aux);
            result:=num;
        end;
    end;
end;
```

VER EL PRIMER ELEMENTO DE UNA PILA

```
function primero_pila (p:tipopila):integer;
begin
    if (not pila_vacia(p)) then
        begin
            result:= p^.dato;
        end;
    end;
end;
```

CONTAR ELEMENTOS DE UNA PILA

```
function contar_pila (p:tipopila):integer;
var
    cont:integer;
begin
    cont:=0;
    while (p <> nil) do
        begin
            cont:= cont+1;
            p:= p^.sig;
        end;
    result:= cont;
end;
```

EJERCICIOS EN PASCAL IMPLEMENTACION ESTÁTICA Y DINÁMICA DE LISTAS, PILAS Y COLAS

1.- {creacion de una lista con N-nodos y metodo de inserccion por el final}

```
program lista1(i,o);
uses
crt;
type
puntero=^nodo;
nodo=record
info:integer;
sig:puntero;
end;
var
lista, aux:puntero;
n,i :integer;
begin
clrscr;
gotoxy (2,2); write ('ingrese largo de la lista: ');
gotoxy (30,2); read(n);
new(lista); {crea el primer nodo}
lista^.info:=1;
lista^.sig:=NIL;
for i:=2 to n do {crea los nodos siguientes}
begin
new(aux); {crea el resto pero con el auxiliar }
aux^.info:=i; {traspaza la informacion }
aux^.sig:=lista;
lista:=aux;
end;
{iniciar el recorrido de la lista}
aux:=lista;
while aux <> nil do
begin
writeln(aux^.info);
aux:=aux^.sig;
end;
repeat until keypressed;
readln;
end.
```

2.- {creacion de una lista con N-nodos por el principio de ella}

```
buscar un elemento en la lista}
program lista1(i,o);
uses
crt;
type
puntero=^nodo;
nodo=record
info:integer;
```

```

sig:puntero;
end;
var
lista, aux:puntero;
n,i,ele :integer;

procedure busqueda(comienzo:puntero;elemento:integer);
var
aux:puntero;
inc:boolean;
begin
aux:=comienzo;
inc:=false;
while (not inc) and (aux <> nil ) do
begin
if aux^.info=elemento then
inc:=true
else
aux:=aux^.sig;
end;
if inc=false then
writeln('NO encontrado')
else
writeln('Encontrado');
end;

{programa principal}
begin
clrscr;

gotoxy (2,2); write ('ingrese largo de la lista: ');
gotoxy (30,2); read(n);
new(lista); {crea el primer nodo}
lista^.info:=1;
lista^.sig:=NIL;
for i:=n downto 2 do {crea los nodos siguientes}
begin
new(aux); {crea el resto pero con el auxiliar }
aux^.info:=i; {traspasa la informacion }
aux^.sig:=lista;
lista:=aux;
end;
gotoxy(5,10);writeln('Ingrese Elemento de Busqueda');
gotoxy(40,10);readln(ele);
busqueda(ele);
readln;
end.

```

3.- BUSCA ELEMENTO EN UNA LISTA

Program lista_nueva(i,o);

```

uses crt;
type
  puntero=^nodo;
  nodo=record
    info:integer;
    sig:puntero;
  end;

```

```

Procedure busqueda(comienzo:puntero;elem:integer; var pos:puntero);
var
  aux:puntero;
  enc:boolean;
begin
  aux:=comienzo;
  enc:=false;
  While (not enc) and (aux<>nil) do
  begin
    If aux^.info=elem then
      enc:=true
    else
      aux:=aux^.sig;
    end;
  If enc then
    write('encontrado');

```

```

  end;
var
  lista,aux,mipos:puntero;
  i,nro:integer;
begin
  new(lista);
  lista^.info:=1;
  aux:=lista;
  For i:=2 to 10 do
  begin
    new(aux^.sig);
    aux:=aux^.sig;
    aux^.info:=i;
  end;
  aux^.sig:=nil;
  write('Elemento a buscar');
  readln(nro);
  busqueda(lista,nro,mipos);
end.

```

4.- { program que busca un elemento y lo borra}

```

program listas4(i,o);
uses
  crt;
type lista=^nodo;

```

```

nodo=record
clave:integer;
sig:lista;
end;

var
i,ene :integer;
lis,aux,lst :lista;
{*****}
procedure borrar(var l:lista;elem:integer);
var
actual,anterior:lista;
begin
actual:=l;
anterior:=l;
while (actual <> nil) and (actual^.clave <> elem) do
begin
anterior:=actual;
actual:=actual^.sig;
end;
if (actual <> nil) and (actual^.clave=elem) then
begin
if (anterior=actual) then
l:=actual^.sig
else
anterior^.sig:=actual^.sig;
gotoxy(5,20);writeln('elemento borrado');
readln;
dispose(actual);
end;
end;
{*****}
procedure muestra(var lst:lista);
begin
aux^.sig:=nil;
aux:=lst;
while aux<> nil do
begin
writeln (aux^.clave);
aux:=aux^.sig;
end;
end;
{***** programa principal *****}
begin
clrscr;
new(lis);
lis^.clave:=1;
aux:=lis;
for i:=1 to 5 do
begin

```

```

new(aux^.sig);
aux:=aux^.sig;
aux^.clave:=l;
end;
aux^.sig:=nil;
{enviar al procedimiento borrar lista}
gotoxy(5,15);writeln('Ingrese Elemento a buscar');
gotoxy(30,15);readln(ene);
lst:=lis;
borrar(lst,ene);
MUESTRA(LIS);
READLN;
end.

```

5.-{crea colas dinámicas}

```

program colas_di(i,o);
uses
crt;
type puntero=^nodo;
nodo=record
dato:integer;
sig:puntero
end;
type
tipocola=record
frente,final:puntero
end;
var
cola,aux:tipocola;
i:integer;

{inicializar cola}
procedure inicola(var cola:tipocola);
begin
cola.frente:=nil;
cola.final:=nil
end;

{cola vacia}
function cola_vacia(col:tipocola):boolean;
begin
cola_vacia:=col.frente=col.final
end;

{insertar un elemento}
procedure insertar(var cola:tipocola;elem:integer);
var
aux:puntero;
begin
if cola_vacia(col) then

```

```

begin
new(aux);
aux^.dato:=elem;
cola.final^.sig:=aux;
cola.final:=aux;
end
else
begin
new(aux);
aux^.dato:=elem;
cola.final:=aux
end;
end;
{ ***** programa principal ***** }
begin
clrscr;
for i:=1 to 10 do
insertar(cola,i);
{impresion}
{ for i:=1 to 10 do
begin
write(aux^.dato);
aux^.final:=aux;
aux^.sig:=aux
end;}
readln;
end.

```

6.- {PROGRAMA CREAR COLAS en forma estatica }

```

PROGRAM COLAS (I,O);
USES
CRT;

```

```

CONST MAX=5;
TYPE TIPOCOLA=RECORD
DATOS :ARRAY[1..5] OF INTEGER;
FRENTE,FINAL:1..MAX;
END;

```

```

VAR
COLA :TIPOCOLA;
ELEM1, I :INTEGER;

```

```

{inicializar cola}
PROCEDURE INICOLA(VAR COLA:TIPOCOLA);
BEGIN
COLA.FRENTE:=MAX;
COLA.FINAL:=MAX;
END;

```

```

{insertar elementos en una cola }
PROCEDURE INSERTAR(VAR COLA:TIPOCOLA;ELEM:INTEGER);
VAR
SIGUIENTE:ARRAY [1..MAX] OF INTEGER;
BEGIN
COLA.FINAL:=COLA.FINAL+1;
COLA.DATOS[COLA.FINAL]:=ELEM
END;

{cola vacia}
function cola_vacia(cola:tipocola):boolean;
begin
cola_vacia:=cola.frente=cola.final
end;

{cola llena}
function cola_llena(COLA:tipocola):boolean;
var
sig:1..max;
begin
if COLA.FINAL=max then
cola_llena:=true
else
sig:=cola.final+1
end;
{***** PROGRAMA PRINCIPAL ***** }
BEGIN
CLRSCR;
GOTOXY(5,5);WRITELN('INGRESE ELEMENTO');
FOR I:=1 TO 5 DO
BEGIN
GOTOXY(30,5);READLN(ELEM1);
INSERTAR(COLA,ELEM1);
END;
READLN;

{impresion cola}
FOR I:=1 TO 5 DO
BEGIN
write(cola.datos[i]);
COLA.FINAL:=COLA.FINAL+1;
END;
readln;
END.

```

7.- {programa que trabaja con 1 pila}

```

Program Pilas;
uses
Crt;
Type TipoPila= ^Nodo;

```

```

Nodo = Record
nueva:integer;
vieja:integer;
Sig :TipoPila ;
end;
var
aux1,aux2,pila1 :TipoPila;
x,num1,num2,c :Integer;

PROCEDURE vacia_pila(var pila:TIPOPILA);
BEGIN
PILA:=NIL;
GOTOXY(15,15);WRITELN('PILA VIEJA VACIA ');
END;

```

```

Function Pila_Vacia(var Pila1:TipoPila):Boolean ;
begin
Pila_vacia:=pila1=Nil; {verifica si la pila esta vacia}
end;

```

```

Procedure Insertar(var pila1:TipoPila; elem1,elem2:integer);
var aux: TipoPila;
begin
if pila_vacia(pila1) then
begin
New(aux);
aux^.nueva:= elem1;
aux^.vieja:=elem2;
aux^.sig:= Pila1;
Pila1:= aux;
end
else
begin
New(aux);
aux^.nueva:= elem1;
aux^.vieja:=elem2;
aux^.sig:= Pila1;
Pila1:= aux;
end;
end;
end;

```

```

{programa principal }
begin
clrscr;
gotoxy(5,5);writeln('Nueva');
gotoxy(5,6);writeln('Vieja');
new(pila1); { ingresa por primera vez }
gotoxy(25,5);readln(pila1^.nueva);
gotoxy(25,6);readln(pila1^.vieja);
pila1^.sig:=nil;

```

```

for x:= 1 to 4 do
begin
gotoxy(25,5);readln(num1);
gotoxy(25,6);readln(num2);
insertar(pila1,num1,num2);
end;
{imprimo pila con sus dos elementos}
aux1:=pila1;
c:=5;;
while aux1 <> nil do
begin
gotoxy(c,10);writeln(aux1^.nueva);
gotoxy(c,14);writeln(aux1^.vieja);
aux1:=aux1^.sig;
c:=c+3;
end;
readln;
aux1:=pila1;
while aux1 <> nil do
begin
aux1^.vieja:=aux1^.nueva;
aux1:=aux1^.sig;
c:=c+3;
end;
aux1:=pila1;
c:=5;;
while aux1 <> nil do
begin
gotoxy(c,16);writeln(aux1^.nueva);
gotoxy(c,18);writeln(aux1^.vieja);
aux1:=aux1^.sig;
c:=c+3;
end;
readln;
end.

```